

COMPRENDRE

Comprendre le projet de bout en bout

Projet Intégrateur 1 — Vidéoprotection intelligente Détection des véhicules éloignés, de petite taille et dans les embouteillages AMA, Cohorte 2, Groupe 5 — Aïchatou TRAORE, Benoît DJOSSOU, Andréa AFOUDA

Comment lire ce document

Il est écrit pour qu'à la fin, vous puissiez **expliquer le projet à quelqu'un qui n'y connaît rien**, puis répondre à un jury technique. C'est le test de Feynman : si vous ne savez pas l'expliquer simplement, c'est que vous ne l'avez pas compris.

Le document est en trois parties, à lire dans l'ordre :

Partie	Contenu	À qui elle sert
I. Les concepts, expliqués depuis zéro	Ce qu'est une image pour un ordinateur, jusqu'à ce qu'est un mAP	À comprendre, pas à réciter
II. Ce que nous avons réellement fait	La procédure, les choix, les chiffres du dépôt	À maîtriser mot pour mot
III. Ce que nous devons encore faire	L'état réel du projet et ce qui manque	À faire avant la soutenance

Chaque concept de la partie I suit le même schéma :

En une phrase. La version que vous diriez à votre petit frère. **L'analogie.** L'image mentale qui la rend évidente. **La version technique.** Le vocabulaire exact, celui du jury. **Pourquoi ça compte ici.** Le lien avec notre projet précis.

PARTIE 0 — Le projet en une page

Avant les détails, la vue d'ensemble. Si vous ne reteniez qu'une page, ce serait celle-ci.

Le contexte. L'État béninois a budgétisé un système national de vidéoprotection dans cinq villes et aux frontières (Conseils des Ministres de mars et juin 2026). Le sujet AMA demande un pipeline complet de vidéo-verbalisation : détecter les véhicules, les suivre, lire les plaques, qualifier une infraction, restituer dans un tableau de bord.

Ce que nous avons choisi de traiter. L'étape 1 du pipeline, et seulement elle : **la détection des véhicules**. Et à l'intérieur de cette étape, le point précis qui casse en contexte béninois : les véhicules **éloignés, petits, ou masqués dans les embouteillages**. Un zémidjan à quarante mètres, c'est un objet de vingt pixels de côté. Aucun système de verbalisation ne fonctionne si la brique de détection rate un véhicule sur deux.

Notre hypothèse. Un modèle YOLOv8 pré-entraîné sur COCO — un jeu de photos grand public, occidental, avec des objets bien cadrés — sous-performe sur du trafic urbain dense. Le ré-entraîner (*fine-tuning*) sur des images de trafic dense réel doit améliorer la détection, **et particulièrement celle des petits objets**.

Notre méthode. Une seule expérience, mais menée proprement :

1. Prendre un jeu public de trafic urbain dense annoté (BMD-45, Bengaluru).
2. Ramener ses 14 classes à 4 classes alignées sur COCO (voiture, moto, bus, camion).
3. Mesurer YOLOv8 pré-entraîné sur un jeu de test isolé.
4. Fine-tuner YOLOv8 sur le train.
5. Re-mesurer sur **exactement le même** jeu de test.
6. Rapporter l'écart, **ventilé par taille d'objet**.

Ce qui fait la valeur du travail. Pas le chiffre final : le **protocole**. Trois précautions précises, détaillées en partie II, font que notre comparaison est honnête là où beaucoup de projets étudiants comparent des choses incomparables sans le savoir.

La limite que nous annonçons nous-mêmes. Il n'existe aucun jeu de données public de vision par ordinateur spécifique au Bénin. Nous travaillons donc sur du trafic urbain dense indien, choisi pour sa proximité de conditions (caméras CCTV fixes, densité extrême, forte proportion de deux-roues). Nous ne prétendons pas avoir « adapté YOLOv8 au trafic béninois »

: nous avons mesuré le gain d'un fine-tuning sur trafic dense, et la validation sur données béninoises est la suite naturelle du travail.

PARTIE I — LES CONCEPTS, EXPLIQUÉS DEPUIS ZÉRO

1. Une image, pour un ordinateur

En une phrase. Une image, c'est un très grand tableau de nombres.

L'analogie. Imaginez une feuille de papier quadrillé géante. Chaque petit carreau porte trois nombres entre 0 et 255 : combien de rouge, combien de vert, combien de bleu. Une photo de 1920 × 1080 pixels, c'est 1920 × 1080 × 3 = 6,2 millions de nombres. L'ordinateur ne voit pas « une voiture » : il voit six millions de nombres. Tout le problème de la vision par ordinateur tient dans une seule question : **comment passe-t-on de six millions de nombres à la phrase « il y a une voiture ici » ?**

La version technique. Une image est un tenseur de forme (hauteur, largeur, canaux) . Les canaux sont RGB (rouge, vert, bleu). OpenCV, la bibliothèque utilisée par Ultralytics, les range dans l'ordre **BGR** et non RGB — d'où l'inversion [:, :, ::-1] que vous verrez dans le code de l'application Streamlit. Un oubli de cette inversion donne des images bleutées.

Pourquoi ça compte ici. Un véhicule éloigné occupe peu de carreaux. S'il fait 20 × 20 pixels dans une image de 1920 × 1080, il représente 0,02 % de l'image. C'est le cœur physique de notre problème : **il y a objectivement peu d'information disponible.**

2. Classification, détection, segmentation

En une phrase. Classifier, c'est dire *quoi* ; détecter, c'est dire *quoi et où*.

L'analogie. Sur une photo de classe :

- la **classification** répond « c'est une photo de classe » ;
- la **détection** répond « il y a un élève ici, un ici, un ici » en dessinant un cadre autour de chacun ;
- la **segmentation** répond pareil, mais en détournant chaque élève au pixel près, comme un découpage aux ciseaux.

La version technique. La détection d'objets produit, pour chaque objet trouvé : une **boîte englobante** (*bounding box*), une **classe**, et un **score de confiance** entre 0 et 1.

Pourquoi ça compte ici. Nous faisons de la **détection**. Nous ne faisons pas de segmentation (trop coûteuse, inutile ici), ni de suivi (*tracking*), ni de lecture de plaque : ces briques sont hors de notre périmètre, même si le sujet AMA les mentionne. Savoir dire *ce qu'on n'a pas fait, et pourquoi* est aussi important que savoir dire ce qu'on a fait.

3. Une boîte englobante, et ses deux écritures

En une phrase. Un rectangle autour d'un objet, écrit soit par ses coins, soit par son centre et sa taille.

Les deux formats à connaître :

Format	Écriture	Qui l'utilise
xyxy	x1, y1, x2, y2 — coin haut-gauche, coin bas-droit, en pixels	Le calcul d'IoU, l'affichage
YOLO	classe cx cy w h — centre, largeur, hauteur, normalisés entre 0 et 1	Les fichiers de labels .txt

Un fichier de labels YOLO ressemble à ceci — une ligne par objet :

```
1 0.487500 0.612963 0.031250 0.055556
0 0.221354 0.545370 0.104167 0.081481
```

Première ligne : un objet de classe 1 (moto chez nous), centré à 48,75 % de la largeur et 61,3 % de la hauteur, faisant 3,1 %

de la largeur de l'image. Sur une image 1920 × 1080, cela fait une boîte de 60 × 60 pixels.

Pourquoi la normalisation ? Parce qu'elle rend le label indépendant de la taille de l'image : si on redimensionne la photo, les coordonnées restent justes. C'est pratique — mais c'est exactement ce qui nous a créé un piège, détaillé au concept 13.

4. Le réseau de neurones convolutif (CNN)

En une phrase. Une machine qui apprend toute seule quels motifs chercher dans une image, en les cherchant partout de la même façon.

L'analogie du tampon. Imaginez un petit tampon de 3 × 3 carreaux que vous faites glisser sur toute l'image, case par case. À chaque position, vous comparez ce que voit le tampon avec un motif : par exemple « clair à gauche, sombre à droite ». Si ça correspond, vous notez un score élevé. En glissant le tampon partout, vous produisez une nouvelle image — une **carte** qui dit « il y a un bord vertical ici, ici et ici ».

C'est ça, une **convolution**. Le tampon s'appelle un **filtre** ou **noyau**, et ses 9 valeurs sont **appprises**, pas choisies par un humain.

L'empilement, qui est la vraie idée. Un seul tampon détecte des bords. Mais si on applique un deuxième tampon sur la carte produite par le premier, il détecte des motifs de bords : des coins, des angles. Un troisième détecte des motifs de coins : des roues, des vitres. Un quatrième : des voitures.

La hiérarchie apprise : pixels → bords → coins et textures → parties d'objets (roue, pare-brise, guidon) → objets (voiture, moto).

Personne n'a programmé « une roue est un cercle sombre ». Le réseau l'a découvert en voyant des centaines de milliers d'images annotées.

Pourquoi glisser le même tampon partout ? Parce qu'une roue est une roue, qu'elle soit en haut à gauche ou en bas à droite. C'est le principe d'**invariance par translation**, et c'est ce qui rend les CNN infiniment plus efficaces qu'un réseau qui traiterait chaque pixel indépendamment.

Le point crucial pour nous : le sous-échantillonnage. Pour que les couches profondes voient « en grand », le réseau réduit régulièrement la taille des cartes : 640 × 640 devient 320, puis 160, puis 80, puis 40, puis 20. On appelle **stride** (pas) le facteur de réduction cumulé. Une carte de stride 32 est 32 fois plus petite que l'image.

Et voilà le drame :

Un véhicule de **20 × 20 pixels** dans l'image d'entrée occupe **moins d'un pixel** sur une carte de stride 32. Il a physiquement disparu.

Pourquoi ça compte ici. Ce paragraphe est la **justification théorique de tout notre projet**. Les petits objets sont difficiles non par accident, mais par construction : l'architecture même du réseau les efface. C'est exactement ce qu'un jury attend d'entendre quand il demande « pourquoi les petits objets sont-ils difficiles ? ».

5. YOLO : « You Only Look Once »

En une phrase. Au lieu d'examiner mille bouts d'image un par un, on regarde l'image **une seule fois** et on prédit tout d'un coup.

L'analogie. Deux façons de compter les personnes dans une salle :

- **L'ancienne méthode (R-CNN, deux étapes).** Vous découpez mentalement la salle en 2 000 zones, et pour chacune vous vous demandez « y a-t-il quelqu'un ici ? ». C'est précis mais interminable.
- **YOLO (une étape).** Vous balayez la salle du regard une fois et vous dites directement « une personne là, là, là ». Plus rapide, et avec un bon œil, presque aussi précis.

La version technique. YOLO découpe l'image en une grille. Chaque cellule est responsable des objets dont le centre tombe dedans, et prédit directement : les coordonnées de la boîte, la classe, et un score. Un seul passage avant (*forward pass*) du réseau produit toutes les détections. D'où le nom, et d'où la vitesse : on parle de dizaines à centaines d'images par seconde.

Pourquoi ça compte ici. Le sujet parle de flux vidéo de caméras de rue. Un système qui met 3 secondes par image ne sert à rien sur un flux à 25 images par seconde. **La vitesse n'est pas un luxe, c'est une contrainte du cas d'usage.**

C'est l'argument central de la diapositive 4 du rapport technique : YOLOv8 offre le meilleur compromis précision/rapidité, et c'est pour cela qu'on l'a retenu plutôt que Faster R-CNN, plus précis mais bien plus lent.

6. Ce qu'il y a dans YOLOv8, précisément

Cette section est celle qui vous distingue si un membre du jury creuse. Elle n'est pas obligatoire pour comprendre la suite.

Trois blocs.

1. **Backbone (l'épine dorsale).** L'empilement de convolutions décrit au concept 4. Il transforme l'image en cartes de caractéristiques de plus en plus abstraites et de plus en plus petites.
2. **Neck (le cou).** Une structure en pyramide (**FPN/PAN**) qui **remélange** les cartes de différentes tailles. C'est important : elle réinjecte l'information sémantique des couches profondes (« ça ressemble à un véhicule ») dans les cartes à haute résolution (« et c'est ici, précisément »).
3. **Head (la tête).** Trois têtes de détection, une par échelle.

Les trois échelles, et pourquoi elles existent. À une entrée de 640 × 640 :

Carte	Stride	Grille	Spécialisée dans
P3	8	80 × 80	Les petits objets
P4	16	40 × 40	Les objets moyens
P5	32	20 × 20	Les grands objets

À retenir. Les petits véhicules de notre sujet sont détectés presque exclusivement par **P3**, la carte de stride 8. Un objet de 20 px y occupe 2,5 pixels. C'est peu, mais ce n'est plus zéro. Voilà pourquoi la **résolution d'entrée** est le paramètre le plus critique de notre projet, et pourquoi la procédure dit de réduire le nombre d'epochs plutôt que `taille_image` si le temps manque : baisser la résolution reviendrait à saboter précisément ce qu'on cherche à mesurer.

Deux spécificités de la v8 par rapport aux versions antérieures :

- **Anchor-free.** Les anciennes versions portaient de rectangles types (*ancres*) prédéfinis et prédisaient des corrections. YOLOv8 prédit directement la position des bords depuis le centre de la cellule. Moins d'hyperparamètres, meilleure généralisation à des formes inhabituelles — ce qui aide sur les deux-roues, dont le rapport hauteur/largeur n'a rien à voir avec celui d'une voiture.
- **Tête découplée.** « Où est l'objet » et « quel objet est-ce » sont prédits par deux branches séparées, plutôt qu'en un seul bloc. Les deux tâches n'entrent plus en concurrence, ce qui améliore les deux.

Les tailles disponibles. YOLOv8 existe en n (nano), s (small), m, l, x. Nous utilisons **yolov8n** : ~3,2 millions de paramètres, 6,5 Mo de poids. C'est le plus petit. Ce choix est assumé et défendable :

- il tourne sur CPU, donc chacun peut le faire tourner sans GPU ;
- il correspond à la contrainte « coût soutenable pour un pays à ressources limitées » explicitement énoncée dans le sujet ;
- et surtout, **la comparaison reste valide** : nous comparons yolov8n à yolov8n. Ce que nous mesurons est l'effet du fine-tuning, pas l'effet de la taille du modèle. Utiliser un modèle plus gros donnerait de meilleurs chiffres absolus mais ne changerait rien à ce que nous voulons démontrer.

7. L'IoU : mesurer si deux rectangles se ressemblent

En une phrase. L'IoU, c'est le pourcentage de recouvrement entre la boîte prédite et la vraie boîte.

L'analogie. Deux personnes dessinent au marqueur un rectangle autour de la même voiture sur une photo. Les rectangles ne coïncident jamais exactement. Comment décider s'ils désignent la même chose ?

On regarde la surface commune aux deux rectangles, et on la divise par la surface totale couverte par au moins l'un des deux.

- Rectangles identiques → IoU = 1
- Rectangles disjoints → IoU = 0
- « Globalement d'accord » → IoU autour de 0,5 à 0,7

La formule :

$$\text{IoU}(A, B) = \frac{\text{aire de l'intersection de A et B}}{\text{aire de l'union de A et B}}$$

Dans notre code, elle est implémentée à la main dans `evaluate.py`, fonction `iou_matrice` : on calcule les coordonnées de l'intersection avec des `maximum` et `minimum`, on en déduit l'aire de l'intersection, et l'union est `aire_A + aire_B - intersection`.

Pourquoi ça compte ici. L'IoU est **l'arbitre de tout le projet**. C'est lui qui décide si une détection compte comme « bonne ». Changer le seuil d'IoU change tous les chiffres. Nous utilisons **IoU $\geq 0,5$** pour le calcul du rappel par taille : une détection qui recouvre au moins la moitié de la vraie boîte compte comme trouvée.

8. Vrais positifs, faux positifs, faux négatifs

En une phrase. Trois façons pour une prédiction d'être juste ou fausse.

L'analogie du pêcheur. Un pêcheur veut attraper les poissons d'un lac.

Cas	Nom	Dans notre projet
Il attrape un poisson	Vrai positif (VP)	Une détection qui correspond à un vrai véhicule (IoU $\geq$ seuil)
Il remonte une vieille botte	Faux positif (FP)	Le modèle dessine une boîte là où il n'y a rien
Un poisson passe à travers le filet	Faux négatif (FN)	Un véhicule réel que le modèle n'a pas vu

Le quatrième cas, absent volontairement. Le « vrai négatif » (l'eau vide correctement ignorée) n'a pas de sens en détection : il y a une infinité de rectangles vides dans une image. C'est pourquoi on n'utilise **jamais l'accuracy** en détection d'objets — question piège classique en soutenance.

9. Précision et rappel : les deux erreurs opposées

En une phrase. La précision demande « ce que j'ai trouvé est-il juste ? », le rappel demande « ai-je trouvé tout ce qu'il y avait ? ».

$$\text{Précision} = \frac{\text{VP}}{\text{VP} + \text{FP}} \quad \text{Rappel} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

L'analogie, suite. Le pêcheur peut tricher dans les deux sens :

- **Précision maximale** : il ne remonte que ce dont il est absolument certain. Il ramène 3 poissons, 3 poissons. Précision = 100 %. Mais il en reste 200 dans le lac : rappel = 1,5 %.
- **Rappel maximal** : il vide le lac à l'épuisette. Il a tous les poissons — rappel = 100 % — mais aussi 400 kg d'algues et deux pneus : précision effondrée.

Les deux se règlent avec un seul bouton : le seuil de confiance. Le modèle sort un score entre 0 et 1 pour chaque boîte. Baisser le seuil, c'est accepter plus de détections : le rappel monte, la précision descend. C'est exactement le curseur « seuil de confiance » de l'application Streamlit, et c'est une très bonne démonstration à faire devant un jury : on déplace le curseur, des motos lointaines apparaissent, et quelques fantômes avec.

Le F1-score réconcilie les deux en une seule note, par moyenne harmonique :

$$\text{F1} = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

La moyenne harmonique, contrairement à la moyenne classique, **punit le déséquilibre** : précision 100 % et rappel 0 % donne F1 = 0, pas 50 %. On ne peut pas tricher en optimisant une seule des deux.

Pourquoi ça compte ici, et c'est fondamental. Notre problématique est un problème de **rappel**, pas de précision. « Les véhicules éloignés ne sont pas détectés » = des faux négatifs. Un jury peut demander « pourquoi insistez-vous sur le rappel

? » : parce que rater un véhicule est le mode d'échec du sujet. Dans un système de vidéo-verbalisation, un véhicule non détecté est un véhicule qui échappe au contrôle.

10. AP et mAP : la note finale

En une phrase. Le mAP résume en un seul nombre la qualité d'un détecteur, **à tous les seuils de confiance à la fois.**

Le problème qu'il résout. Précision et rappel dépendent du seuil de confiance choisi. Comparer deux modèles à un seuil arbitraire, c'est comparer deux coureurs sur une distance choisie au hasard. On veut une note qui ne dépende pas de ce choix.

La construction, étape par étape :

1. On prend **toutes** les détections du modèle, y compris les très peu sûres — d'où le `conf=0.001` dans le code d'évaluation, qui surprend au premier abord.
2. On les trie de la plus confiante à la moins confiante.
3. On les parcourt dans cet ordre. À chaque détection ajoutée, on recalcule précision et rappel courants et on place un point sur un graphe.
4. On obtient la **courbe précision-rappel**, qui part en haut à gauche (peu de détections, très sûres) et descend vers la droite (beaucoup de détections, moins sûres).
5. **L'AP (Average Precision) est l'aire sous cette courbe.** Une aire proche de 1 signifie « le modèle reste précis même quand on lui demande de tout trouver ».
6. On calcule une AP par classe, et le **mAP (mean AP)** en est la moyenne sur les classes.

Les deux variantes que nous rapportons :

Métrique	Définition	Ce qu'elle dit
mAP@0.5	AP avec IoU $\geq 0,5$	« Le modèle voit-il les véhicules ? » Tolérant sur le cadrage
mAP@0.5:0.95	Moyenne des AP pour IoU = 0,50 ; 0,55 ; ... ; 0,95	« Le modèle les cadre-t-il précisément ? » Exigeant

La seconde est toujours nettement plus basse. C'est normal, ce n'est pas une contre-performance : à IoU 0,95, il faut que la boîte prédite épouse presque parfaitement la vraie. C'est la métrique standard du benchmark COCO, et c'est celle citée dans les publications.

Pourquoi ça compte ici. Le mAP est la métrique que tout le monde attend, donc nous la donnons. Mais — point capital — **le mAP ne répond pas à notre question.** Il moyenne sur toutes les tailles d'objets. Un modèle peut gagner 5 points de mAP uniquement parce qu'il détecte mieux les bus au premier plan, et n'avoir rien amélioré sur les motos du fond. C'est précisément pour cela que nous avons ajouté la mesure du concept 13.

11. La NMS : supprimer les doublons

En une phrase. Quand le modèle dessine cinq boîtes sur la même voiture, on ne garde que la meilleure.

L'analogie. Cinq témoins décrivent le même accident. On ne rédige pas cinq procès-verbaux : on garde la déposition la plus fiable et on écarte celles qui racontent la même scène.

La version technique. *Non-Maximum Suppression* : on trie les détections par score décroissant ; on garde la meilleure ; on supprime toutes celles qui ont un IoU supérieur à un seuil avec elle ; on recommence avec la suivante.

Pourquoi ça compte ici, et c'est subtil. Dans un embouteillage, **deux véhicules réels différents se recouvrent beaucoup.** Une NMS trop agressive supprimera la seconde moto en croyant supprimer un doublon. C'est une cause directe et documentée des pertes de détection en trafic dense — exactement la troisième puce de notre diapositive « limites observées ». Savoir nommer ce mécanisme est un très bon point en soutenance.

12. Le transfert d'apprentissage et le fine-tuning

En une phrase. Plutôt que de tout réapprendre depuis zéro, on part d'un modèle qui sait déjà voir et on lui apprend

nos rues.

L'analogie. Vous voulez former quelqu'un à reconnaître les véhicules de Cotonou. Deux options :

- **Depuis zéro.** Prendre un nouveau-né et lui apprendre d'abord ce qu'est une forme, un contour, une ombre, une roue... Il faudrait des millions d'exemples et des semaines de calcul.
- **Par transfert.** Prendre un adulte qui sait déjà voir — il a grandi ailleurs, il connaît les voitures européennes — et lui montrer quelques milliers de photos de Cotonou. En quelques heures il s'adapte.

Le fine-tuning, c'est la seconde option. Les premières couches du réseau, celles qui détectent bords et textures, sont **universelles** : un bord est un bord partout dans le monde. Ce qui doit changer, ce sont les couches finales, celles qui décident « ceci est une moto, dans ce contexte, à cette échelle, dans cette densité ».

Concrètement dans notre code :

```
modele = YOLO("models/yolov8n.pt") # on charge les poids appris sur COCO
modele.train(data="configs/dataset.yaml", epochs=50, ...)
```

Ces deux lignes *sont* le fine-tuning. On ne repart pas de poids aléatoires : on repart des poids COCO et on continue l'entraînement sur nos données.

Pourquoi ça compte ici. C'est **la contribution technique du projet**. Toute la comparaison consiste à mesurer ce que ces deux lignes apportent.

13. Le décalage de domaine (*domain gap*)

En une phrase. Un modèle entraîné sur un type d'images se dégrade sur un autre type d'images, même si les objets sont les mêmes.

L'analogie. Quelqu'un qui a appris à conduire uniquement sur autoroute allemande, propre et balisée, se retrouve sur le carrefour de Dantokpa à 17 heures. Il sait toujours conduire — mais tout ce qu'il a intégré comme « normal » est faux ici : la densité, les trajectoires, les distances.

Le chiffre à connaître par cœur. Publié à CVPR 2026 avec le jeu de données BMD-45 : des détecteurs entraînés sur **UA-DETRAC** (benchmark occidental classique) n'atteignent que **33,6 %** de mAP@0.50:0.95 sur du trafic urbain dense de pays émergent, contre **83,8 %** quand ils sont entraînés sur des données du domaine. **Un facteur 2,5.**

Pourquoi ça compte ici. Ces deux nombres justifient tout le projet en une phrase, et ils vous donnent l'ordre de grandeur de ce que votre propre comparaison peut espérer retrouver. À citer en introduction de la soutenance.

14. La convention de taille COCO — et le piège qui nous est arrivé

En une phrase. Un objet est dit « petit » s'il fait moins de 32×32 pixels, mais **à quelle résolution ?**

La convention COCO, standard du domaine :

Catégorie	Aire de la boîte
petit	$< 32 \times 32 = 1\,024 \text{ px}^2$
moyen	entre $1\,024$ et $96 \times 96 = 9\,216 \text{ px}^2$
grand	$> 9\,216 \text{ px}^2$

Le piège. Ces seuils supposent implicitement des images d'environ 640 px de côté, la taille typique du benchmark COCO. Nos images de trafic sont bien plus grandes. Prenons une image de 2 560 px de large et une moto lointaine qui occupe 2 % de sa largeur :

- **en pixels de l'image d'origine** : $51 \times 51 \text{ px}$, soit $2\,601 \text{ px}^2 \rightarrow$ catégorie « **moyen** », alors que c'est visiblement un petit objet ;
- **ramenée à 640 px** (facteur 0,25) : $13 \times 13 \text{ px}$, soit $169 \text{ px}^2 \rightarrow$ catégorie « **petit** », ce qui correspond à la réalité perçue par le réseau.

Plus l'image d'origine est grande, plus l'effet est violent : sur des photos de plusieurs milliers de pixels, **la quasi-totalité des véhicules bascule dans la catégorie « grand »**, la catégorie « petit » se vide, et la mesure ne mesure plus rien.

C'est exactement ce qui s'est produit lors de notre premier essai. Le README du projet le mentionne noir sur blanc. C'est une anecdote à raconter en soutenance : elle prouve que vous avez regardé vos chiffres au lieu de les recopier.

Notre correction. Avant de classer une boîte, on ramène son aire à la résolution que le réseau voit réellement, 640 px :

```
echelle = TAILLE_REFERENCE / max(largeur, hauteur) # 640 / plus grand côté
aire = aire_en_pixels * echelle**2                # aire ramenée à 640 px
```

Le facteur est au carré parce qu'on transforme une **aire**, pas une longueur : diviser les deux côtés par 2 divise l'aire par 4.

Pourquoi c'est méthodologiquement juste. Le réseau redimensionne l'image à 640 px avant de la traiter. Ce que « voit » réellement le détecteur, c'est l'objet à cette échelle-là. Classer les tailles à la résolution d'entrée du réseau, c'est classer les objets selon la difficulté réelle qu'ils posent au modèle — pas selon la résolution accidentelle de l'appareil photo.

15. L'augmentation de données

En une phrase. Fabriquer artificiellement des variantes de nos images pour que le modèle en voie davantage, et de plus variées.

L'analogie. Pour apprendre à un enfant à reconnaître un chat, on ne lui montre pas 100 fois la même photo. On lui montre des chats de dos, à l'ombre, de loin, en partie cachés. L'augmentation fait cela automatiquement : elle retourne, décale, assombrit, zoome les images d'entraînement.

Nos réglages, un par un — c'est le tableau à savoir défendre :

Réglage	Valeur	Ce qu'il fait	Pourquoi cette valeur
mosaic	1.0	Assemble 4 images en une seule	Chaque image occupe un quart de la surface : tous les objets deviennent 2 fois plus petits . C'est l'augmentation la plus directement liée à notre problématique
scale	0.7	Zoom aléatoire $\pm 70\%$	Un zoom arrière crée des véhicules plus petits que ceux réellement photographiés. Second levier « petits objets »
flipLR	0.5	Miroir horizontal, une image sur deux	Un véhicule vu de gauche ou de droite reste un véhicule. Double gratuitement la variété
flipUD	0.0	Miroir vertical — désactivé	Une scène routière a un haut (ciel) et un bas (bitume). Retourner l'image apprendrait au modèle un monde qui n'existe pas
degrees	0.0	Rotation — désactivée	Nos caméras de vidéoprotection sont fixes . Le modèle n'a aucune raison d'apprendre des scènes penchées
hsv_v	0.4	Variation de luminosité	Plein soleil, ombre d'un manguier, fin de journée : conditions réelles du terrain
hsv_s , hsv_h	0.7 / 0.015	Saturation et teinte	Robustesse aux différences de capteur et de balance des blancs
translate	0.1	Décalage de l'image	Le modèle ne doit pas supposer que les véhicules sont toujours au centre

Le raisonnement d'ensemble, à énoncer tel quel devant un jury : les augmentations activées sont celles qui **produisent des objets plus petits** (mosaic, scale) ou qui reflètent des variations réelles du terrain (luminosité, miroir horizontal). Les augmentations désactivées sont celles qui **fabriqueraient des situations physiquement**

impossibles dans notre cas d'usage (rotation, retournement vertical). Ce n'est pas une liste par défaut, c'est un choix raisonné au regard de la problématique.

16. Le vocabulaire de l'entraînement

Terme	En une phrase	Notre valeur	Pourquoi
Epoch	Un passage complet sur toutes les images d'entraînement	50	Compromis courant entre qualité et temps machine
Batch	Nombre d'images traitées en une fois avant mise à jour des poids	16	Tient dans la mémoire d'un GPU T4 gratuit
Fonction de perte	La note d'erreur que le réseau cherche à diminuer	—	YOLOv8 combine une perte de boîte (CloU), une perte de classe et une DFL
Descente de gradient	La méthode : ajuster chaque poids dans le sens qui diminue l'erreur	—	L'analogie : descendre une colline dans le brouillard en tâtant la pente
Patience / early stopping	Arrêter si N epochs passent sans progrès sur la validation	15	Évite de gaspiller du temps machine et de surapprendre
Graine (seed)	Fixe le hasard : mélanges, initialisations, augmentations	42	Reproductibilité. Sans graine fixe, deux exécutions donnent des chiffres différents et rien n'est comparable
Taille d'image	Résolution à laquelle le réseau travaille	640	Le paramètre le plus critique pour les petits objets (voir concept 6)

Le surapprentissage (overfitting), en une image. Un élève qui apprend les corrigés par cœur : il a 20/20 sur les exercices vus, 4/20 sur un exercice nouveau. Un modèle surappris a des métriques excellentes sur l'entraînement et médiocres ailleurs. C'est exactement ce que la séparation train/val/test sert à détecter.

17. Train, validation, test : trois rôles, jamais mélangés

Split	Rôle	Analogie scolaire	Chez nous
Train	Le modèle apprend dessus	Les exercices d'entraînement	2 400 images
Validation	On surveille l'apprentissage, on arrête au bon moment	Les interrogations blanches	300 images
Test	On mesure une seule fois, à la fin	L'examen final	300 images

La règle absolue : le modèle ne doit **jamais** avoir vu le jeu de test pendant l'entraînement, ni même indirectement via des choix de réglages. Sinon, la note est un mensonge — c'est un examen dont on aurait vu le sujet.

Le piège de la contamination. Beaucoup de jeux de données publics contiennent les mêmes images sources dans plusieurs splits, parfois simplement augmentées. Leurs métriques publiées sont alors surestimées. Repérer et signaler ce défaut est exactement ce qu'un jury technique valorise, bien plus qu'un chiffre élevé.

Notre cas précis. BMD-45 fournit `train` et `val`, mais pas de `test`. Notre script `import_dataset.py` **prélève donc la moitié de la validation pour en faire un test**, de façon déterministe (graine 42, mélange reproductible). D'où nos 600 images de validation devenues 300 val + 300 test. La procédure est explicite là-dessus : « *sans jeu de test isolé, il n'y a pas de résultat, seulement une démonstration* ».

18. Les FPS

En une phrase. Combien d'images le modèle traite par seconde.

Une caméra de rue produit typiquement 25 images par seconde. Un modèle à 8 FPS ne peut pas suivre en temps réel — il faudrait sous-échantillonner le flux, ce qui est acceptable pour du comptage mais problématique pour de l'estimation de

vitesse.

Notre mesure, et sa subtilité. Dans `evaluate.py`, on lance une inférence **à vide avant de démarrer le chronomètre** :

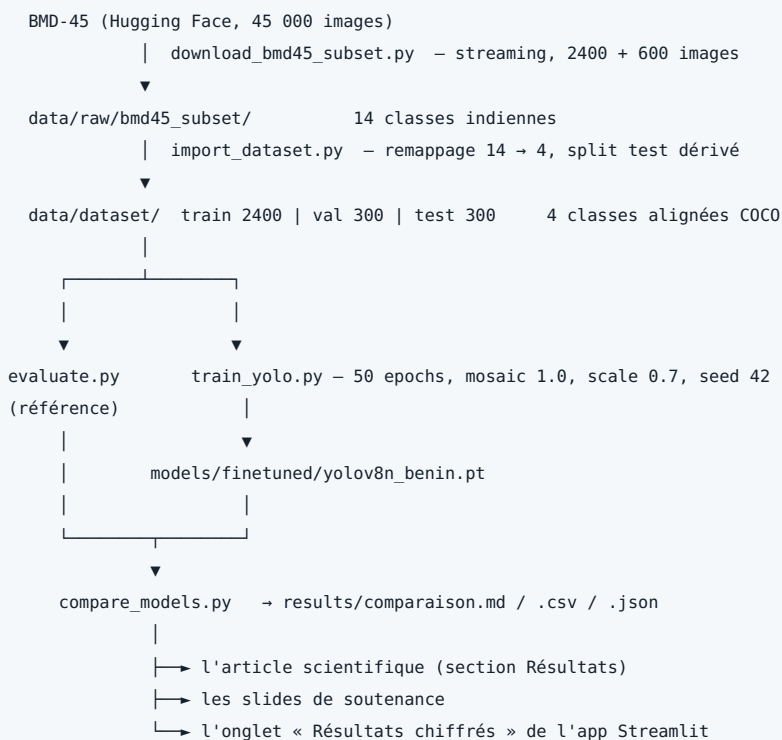
```
# Echauffement : le premier appel paie l'initialisation du modèle
modele.predict(str(images[0]), ...)
```

Le premier appel paie le chargement des noyaux CUDA, l'allocation mémoire, l'initialisation du device. L'inclure dans la mesure fausserait le débit vers le bas. Ce détail de trois lignes est le genre de rigueur qu'un jury remarque.

Attention à ce que vous affirmez. Les FPS mesurés dépendent entièrement de la machine (CPU portable, GPU T4 Colab...). Toujours annoncer le matériel avec le chiffre. Sans matériel, un FPS ne veut rien dire.

PARTIE II — CE QUE NOUS AVONS RÉELLEMENT FAIT

19. La chaîne complète, en une vue



20. Choix n° 1 : réduire le périmètre

Le sujet AMA demandait six étapes : détection, tracking, détection de plaque, OCR, règles d'infraction, tableau de bord.

Nous traitons l'étape 1 seule, et à l'intérieur de celle-ci, un verrou précis. Sont explicitement hors périmètre : la lecture de plaques (ANPR/OCR), le suivi multi-objets et les identifiants persistants, les règles d'infraction et l'estimation de vitesse, la génération de procès-verbaux.

Comment le défendre. Trois arguments, dans cet ordre :

1. **La détection est la brique fondatrice.** Si elle rate un véhicule sur deux, toutes les briques suivantes héritent de l'erreur. Aucun OCR de plaque ne rattrape un véhicule jamais détecté. Améliorer le maillon d'entrée est l'investissement le plus rentable de la chaîne.
2. **Le verrou choisi est celui du contexte béninois.** Forte densité de deux-roues, embouteillages, véhicules éloignés : ce n'est pas un sous-problème arbitraire, c'est *le* mode d'échec local.
3. **Un travail restreint et mesuré vaut mieux qu'un pipeline complet et invérifiable.** Six briques bâclées ne produisent aucun chiffre défendable. Une brique traitée avec un protocole propre en produit.

21. Choix n° 2 : le jeu de données

Le constat de départ, à annoncer soi-même. Il n'existe **aucun jeu de données public de vision par ordinateur spécifique au Bénin** pour la détection de véhicules. Ni sur Roboflow Universe, ni sur Hugging Face, ni dans la littérature.

Conséquences directes sur ce que nous pouvons affirmer :

- nous **ne pouvons pas** mesurer l'écart de domaine *béninois*, faute de données béninoises pour le mesurer ;
- nous **pouvons** mesurer le gain d'un fine-tuning sur du **trafic urbain dense**, en particulier sur les véhicules de petite taille. C'est l'objet du projet.

Le jeu retenu : BMD-45 (`iisc-aim/BMD-45` sur Hugging Face).

Caractéristique	Valeur	Pourquoi c'est le bon choix ici
Volume total	45 000 images, 480 000 boîtes	Largement assez pour en extraire un sous-ensemble
Origine	Plus de 3 600 caméras CCTV opérationnelles , Bengaluru (Inde)	Caméras fixes en hauteur = exactement le cas d'usage vidéoprotection
Classes	14 catégories fines, dont des modes locaux (auto-rickshaws)	Couvre voiture/moto/bus/camion, condition de la comparaison COCO
Publication	CVPR 2026	Référence citable, protocole documenté
Conditions	Densité extrême, occlusions, variation de point de vue	Les trois axes de notre problématique

Les quatre critères qui ont guidé le choix, par ordre d'importance : beaucoup de petits objets (c'est l'objet de l'étude), trafic dense avec chevauchements, prise de vue fixe en hauteur, classes proches de COCO.

Ce que BMD-45 n'est pas. Il est indien, pas béninois. Nous ne le maquillons pas. La formulation à employer :

À éviter : « nous avons adapté YOLOv8 au trafic béninois ».

À dire : « faute de jeu de données béninois public, nous avons travaillé sur du trafic urbain dense d'un contexte comparable. Nous mesurons le gain sur les véhicules de petite taille, qui est le verrou identifié. La validation sur données béninoises est la suite naturelle de ce travail. »

Le sous-ensemble, et pourquoi. 45 000 images sont hors budget en temps machine. `download_bmd45_subset.py` lit le dataset **en streaming** (sans le télécharger en entier ni le charger en mémoire), mélange avec une graine fixée (42 pour le train, 43 pour la validation, afin que les deux flux ne tirent pas les mêmes échantillons), et n'écrit localement que **2 400 images d'entraînement et 600 de validation**.

Deux détails du script qui méritent d'être connus :

- **Reprise après interruption.** Le script compte les paires image+label déjà écrites et reprend là où il s'était arrêté. Une paire incomplète (interruption entre l'écriture de l'image et celle du label) est supprimée pour repartir propre. Sur une connexion instable, c'est ce qui rend le téléchargement faisable.
- **Qualité JPEG à 95.** PIL ré-encode à l'enregistrement avec une qualité par défaut d'environ 75, ce qui **dégraderait visiblement les petits objets** — précisément le cœur du sujet. Le script force 95. Encore un détail à trois lignes qui montre que le sujet a guidé chaque décision.

22. Choix n° 3 : quatre classes alignées sur COCO

Le problème à résoudre. Nous voulons comparer deux modèles :

- le **pré-entraîné COCO** connaît 80 classes, dont `car` (indice 2), `motorcycle` (3), `bus` (5), `truck` (7) ;
- le **fine-tuné** connaît 4 classes : `voiture` (0), `moto` (1), `bus` (2), `camion` (3).

Les mêmes objets portent des numéros différents. Si on évalue naïvement le modèle COCO avec nos labels, l'indice 1 de nos fichiers (« moto ») sera compris comme `bicycle` par le modèle COCO. **Tous les chiffres du pré-entraîné seraient faux**, et faux dans le sens qui nous arrange — ce qui est le pire des cas.

Notre solution, en deux endroits :

À l'import (`import_dataset.py`). Les 14 classes de BMD-45 sont remappées :

Classes BMD-45	Notre classe
Hatchback, Sedan, SUV, MUV, Van	voiture
Two-wheeler	moto

Bus, Mini-bus, Tempo-traveller	bus
Truck, LCV	camion
Three-wheeler, Bicycle, Other	ignorées

Les auto-rickshaws (Three-wheeler) et les vélos sont **ignorés à dessein** : ils n'ont pas d'équivalent parmi nos quatre classes COCO, et les conserver fausserait la comparaison avec un modèle pré-entraîné qui ne sait pas les détecter comme tels.

Le garde-fou. Le script **refuse d'importer** tant qu'une classe source n'est pas explicitement traitée, en cible ou en ignorer . Une classe oubliée fausserait silencieusement toutes les métriques : mieux vaut un script qui s'arrête qu'un résultat faux qu'on ne détecte jamais. Lancé une première fois, il affiche la liste réelle des classes du jeu source.

À l'évaluation (evaluate.py). On ne peut pas remapper le modèle pré-entraîné : ses 80 classes sont figées dans ses poids. On remappe donc **les labels**. La fonction construire_vue_coco crée une copie temporaire du jeu de test dans laquelle nos indices 0/1/2/3 sont réécrits en 2/3/5/7 :

```
LOCAL_VERS_COCO = {0: 2, 1: 3, 2: 5, 3: 7}
```

Deux finesses d'implémentation :

- les **images sont liées en dur** (os.link), pas copiées : la vue est instantanée et ne duplique aucun octet sur le disque ; seuls les petits fichiers texte de labels sont réécrits ;
- la vue est **supprimée après l'évaluation** — c'est un artefact de calcul, pas une donnée du projet.

En une phrase pour le jury : « Les deux modèles sont mesurés sur exactement les mêmes images et les mêmes boîtes de référence ; seule l'étiquette numérique est traduite dans le vocabulaire de chaque modèle. »

Sans ces deux précautions, la comparaison n'a aucune valeur. C'est le point méthodologique n° 1 du projet.

23. Volumétrie réelle du jeu importé

Chiffres mesurés directement sur data/dataset/ du dépôt, à la date de rédaction.

Split	Images	voiture	moto	bus	camion	Total instances
train	2 400	5 744	11 641	1 298	1 873	20 556
val	300	752	1 476	138	215	2 581
test	300	742	1 380	189	240	2 551
Total	3 000	7 238	14 497	1 625	2 328	25 688

Ce tableau est indispensable dans l'article et dans les slides. Une étude sans volumétrie n'est pas évaluable. Trois lectures à en tirer :

1. **Environ 8,5 véhicules annotés par image en moyenne.** C'est la signature objective d'un trafic dense. Ce n'est pas une impression, c'est un chiffre.
2. **Les motos représentent 56 % des instances** (14 497 sur 25 688). C'est l'argument le plus fort de tout le projet pour la transposition au Bénin : la dominance des deux-roues, caractéristique du trafic béninois avec les zémidjans, **est présente dans notre jeu de données**. Ce n'est pas un jeu de trafic occidental déguisé.
3. **Bus et camions sont minoritaires** (1 625 et 2 328 instances, soit 6 % et 9 %). Sur le test, 189 bus et 240 camions : c'est assez pour une métrique indicative, mais il faut **annoncer que les métriques par classe y sont moins stables**. Publier un chiffre calculé sur trois objets serait une faute ; sur deux cents, c'est acceptable à condition de le dire.

24. Choix n° 4 : les hyperparamètres d'entraînement

Tout est dans configs/entrainement.yaml , rien n'est codé en dur dans les scripts. Un run est donc **entièrement décrit par son fichier de configuration**, donc reproductible — c'est ce qui permet à chaque membre du groupe d'obtenir les mêmes chiffres.

```
modele_base: models/yolov8n.pt
epochs: 50
taille_image: 640
batch: 16
patience: 15
seed: 42
```

Justifications, dans l'ordre où on vous les demandera :

- **yolov8n** : le plus petit modèle, tourne sur CPU, cohérent avec la contrainte « coût soutenable ». Comparaison n vs n, donc valide.
- **640 px** : résolution standard de YOLOv8. Le paramètre à ne jamais baisser ici (concept 6).
- **50 epochs** : compromis ; environ une heure sur GPU T4 gratuit pour 2 400 images. La procédure prévoit de descendre à 25 si le temps manque — l'écart serait moindre mais la comparaison resterait valide.
- **patience 15** : arrêt anticipé si la validation ne progresse plus.
- **seed 42** : reproductibilité. Sans elle, deux exécutions donnent des chiffres différents et rien n'est comparable.

Les augmentations sont détaillées au concept 15 — c'est le tableau qu'un jury technique attaquera en premier, parce que c'est là que se joue le lien entre les réglages et la problématique.

Où atterrissent les poids. Ultralytics écrit dans `results/entrainements/yolov8n_benin/weights/best.pt` ; `train_yolo.py` copie automatiquement ce meilleur modèle vers `models/finetuned/yolov8n_benin.pt`, pour que l'évaluation, la comparaison, `detect_image.py` et l'application Streamlit aient un **chemin stable** à viser.

25. Choix n° 5 : le protocole d'évaluation

C'est la partie la plus fine du projet, et celle qui fait la différence.

25.1 Les métriques globales

Calculées par le validateur d'Ultralytics (`modele.val`) :

```
resultats = modele.val(data=..., split=split, conf=0.001, iou=0.6,
                       classes=COCO_VEHICULES if modele_est_coco(modele) else None)
```

Deux paramètres surprennent et doivent être expliqués :

- **conf=0.001** . On accepte quasiment toutes les détections. Ce n'est pas une erreur : pour tracer la courbe précision-rappel et en calculer l'aire (l'AP, concept 10), il **faudrait** toutes les détections, y compris les moins sûres. Un seuil élevé tronquerait la courbe et sous-estimerait l'AP.
- **classes=[2,3,5,7]** **pour le modèle COCO**. On restreint le pré-entraîné aux quatre classes de véhicules. Sans cela, il détecterait des personnes, des feux tricolores, des sacs, qui compteraient tous comme faux positifs et l'écraieraient injustement. **On lui donne les mêmes conditions qu'au modèle fine-tuné**, qui ne connaît que quatre classes.

De ces résultats on tire mAP@0.5, mAP@0.5:0.95, précision, rappel, et le F1 recalculé comme moyenne harmonique.

25.2 Le rappel par taille — la mesure propre au sujet

C'est la seule métrique qui **répond directement à la question posée par le titre du projet**, et elle est calculée à la main dans `evaluate.py` parce qu'Ultralytics ne la fournit pas.

L'algorithme, en français :

1. Pour chaque image du test, lancer une prédiction à **conf=0.25** — cette fois un seuil réaliste, celui d'un usage opérationnel (on veut savoir ce que le système trouverait en production, pas ce qu'il pourrait trouver en ratissant tout).
2. Charger les vraies boîtes du fichier de labels, les convertir en pixels absolus.
3. Calculer la matrice d'IoU entre toutes les vraies boîtes et toutes les boîtes prédites.
4. Pour chaque vraie boîte : calculer son aire, **la ramener à 640 px** (concept 14), la ranger dans petit / moyen / grand.
5. Parcourir les prédictions par IoU décroissant avec cette vraie boîte, et retenir la première qui est à la fois au-dessus de 0,5 et pas déjà attribuée à une autre vraie boîte. Si on en trouve une, l'objet est compté comme détecté. (Appariement glouton : si la meilleure prédiction est déjà prise par une autre boîte, la deuxième meilleure peut

encore valider celle-ci.)

6. Rappel de chaque catégorie = détectés / total.

Quatre décisions à savoir défendre :

Décision	Justification
Seuil de confiance 0,25 (et non 0,001)	On mesure ce qu'un système déployé détecterait réellement
Une prédiction ne valide qu'une seule vraie boîte (déjà_prises)	Sans cette règle, une grosse boîte imprécise pourrait « valider » cinq motos voisines dans un embouteillage. C'est un appariement glouton un-à-un, la convention standard
Rappel agnostique à la classe	On demande « le véhicule a-t-il été vu ? », pas « a-t-il été bien nommé ? ». Confondre une moto et un vélo est une erreur mineure ; ne pas voir le véhicule du tout est le mode d'échec du sujet
Rappel et non précision, par taille	Le rappel répond à « combien de véhicules ai-je ratés ? », qui est la question du projet

25.3 Pourquoi cette mesure est la ligne décisive

Un gain de mAP global peut provenir **entièrement** des gros véhicules au premier plan. Dans ce cas le projet n'a rien résolu, tout en affichant un beau chiffre. Seule la ventilation par taille permet de distinguer :

« le modèle est meilleur » (banal) de « le modèle est meilleur **sur ce qui ne marchait pas** » (notre thèse).

26. Les trois lectures possibles du résultat

C'est peut-être la partie la plus importante de ce document pour votre tranquillité d'esprit : **les trois issues possibles sont toutes présentables**, à condition d'être expliquées.

Ce que vous observerez	Ce que cela signifie	Ce qu'il faut dire
Le rappel des petits progresse nettement, celui des grands peu	Résultat attendu : le fine-tuning a corrigé le point visé	C'est votre résultat principal. Mettez-le en avant, c'est la thèse démontrée
Toutes les tailles progressent de manière comparable	Le gain vient de l'adaptation au domaine (couleurs, cadrage, densité) plus que de la taille	Dites-le ainsi. C'est un résultat honnête et défendable : vous avez mesuré un <i>domain gap</i> , ce qui est exactement le sujet du papier BMD-45
Rien ne progresse	Le jeu ressemble déjà beaucoup aux données COCO, ou le sous-ensemble est trop petit	Rapportez la volumétrie et la nature du jeu. Le diagnostic devient le résultat

La phrase à retenir, elle vaut pour toute la soutenance : un résultat mesuré et expliqué vaut mieux qu'un résultat impressionnant sans protocole. Si les chiffres sont mauvais, présentez-les **avec** le diagnostic. Un jury technique sanctionne un chiffre invérifiable, pas un chiffre modeste bien analysé.

Le troisième cas est plus probable avec un jeu public généraliste qu'avec des images collectées soi-même. Si cela arrive, les pistes de diagnostic à évoquer : sous-ensemble trop petit (2 400 images), modèle nano trop contraint, 50 epochs insuffisants, ou proximité réelle entre BMD-45 et les données d'entraînement de COCO.

27. Les limites du travail, à annoncer avant qu'on les oppose

Une limite énoncée par vous est une preuve de lucidité ; la même limite découverte par le jury est une faille. La liste :

1. **Pas de données béninoises.** Aucun jeu public n'existe. Nous travaillons sur du trafic dense indien, choisi pour sa proximité de conditions (CCTV fixes, densité, dominance des deux-roues). La validation locale est la perspective n° 1.
2. **Sous-ensemble de 3 000 images** sur les 45 000 disponibles. Contrainte de temps machine, assumée. Un sous-ensemble plus grand est la perspective n° 2.
3. **Modèle nano.** Choisi pour la contrainte de coût. Un yolov8s ou m donnerait de meilleurs chiffres absolus, sans changer la conclusion sur l'effet du fine-tuning.
4. **La limite physique.** Au-delà d'une certaine distance, un véhicule occupe moins de N pixels et **aucun modèle ne peut le détecter**. C'est une limite de l'information disponible, pas un défaut de notre approche. Le remède n'est

pas algorithmique : c'est une caméra de plus haute résolution, ou mieux placée.

5. **Classes minoritaires.** 189 bus et 240 camions dans le test : les métriques par classe y sont moins stables que pour voitures et motos.
6. **Contamination train/test possible.** Défaut connu de certains jeux publics. À vérifier sur quelques images et à signaler le cas échéant.
7. **Données personnelles.** Les images de trafic montrent plaques et visages. Le sujet AMA le souligne : la plaque est une donnée à caractère personnel, et tout déploiement réel suppose un cadre juridique explicite (APDP Bénin).
Floutez plaques et visages sur toute capture destinée aux slides ou à l'article.
8. **Licence du jeu.** À vérifier sur la page Hugging Face de BMD-45 et à citer. *Une licence non spécifiée n'est pas une autorisation.*

28. Le dépôt, fichier par fichier

Chemin	Rôle	À connaître
<code>configs/import.yaml</code>	Chemin du jeu source + correspondance des classes	La table de remappage 14 → 4
<code>configs/dataset.yaml</code>	Chemins des splits et noms des 4 classes	Généré automatiquement , ne pas éditer à la main. Chemin absolu volontairement, car Ultralytics résout un chemin relatif par rapport à son réglage global <code>datasets_dir</code> et non par rapport au fichier
<code>configs/entrainement.yaml</code>	Tous les hyperparamètres	La source unique de vérité d'un run
<code>scripts/download_bmd45_subset.py</code>	Sous-ensemble BMD-45 en streaming	Reprise après interruption, JPEG qualité 95
<code>scripts/import_dataset.py</code>	Remappage + garantie d'un test isolé	Refuse d'importer si une classe n'est pas traitée
<code>scripts/train_yolo.py</code>	Fine-tuning	Copie <code>best.pt</code> vers un chemin stable
<code>scripts/evaluate.py</code>	Métriques d'un modèle	Vue COCO temporaire, rappel par taille, échauffement FPS
<code>scripts/compare_models.py</code>	Le tableau comparatif	Le livrable central du projet
<code>scripts/detect_image.py</code>	Visuels qualitatifs	Suffixe le nom du modèle pour les mises côte à côte
<code>notebooks/colab_entrainement.ipynb</code>	Toute la chaîne sur GPU gratuit	Sauvegarde sur Drive, reprise de téléchargement
<code>docs/PROCEDURE.md</code>	Quoi faire, dans quel ordre	Le mode d'emploi
<code>docs/DATASETS.md</code>	Choix du jeu et critères	La justification du choix n° 2
<code>docs/GUIDE_STREAMLIT.md</code>	Construire la démo	Code complet de l'application
<code>docs/GUIDE_ARTICLE_LATEX.md</code>	Rédiger l'article	Trame LaTeX prête à compiler

PARTIE III — OÙ NOUS EN SOMMES, ET CE QU'IL RESTE À FAIRE

29. État réel du dépôt

Constaté directement sur la copie locale :

Élément	État	Commentaire
Environnement, dépendances	✓ Fait	<code>requirements.txt</code> figé, ultralytics 8.4.114, torch 2.13
Sous-ensemble BMD-45 téléchargé	✓ Fait	2 400 + 600 images dans <code>data/raw/</code>
Import et remappage	✓ Fait	2 400 / 300 / 300, volumétrie en section 23
Fine-tuning	✓ Fait	<code>models/finetuned/yolov8n_benin.pt</code>

		présent (6,2 Mo)
Évaluation du pré-entraîné	⚠ Absent en local	results/ est vide
Tableau comparatif	⚠ Absent en local	Aucun comparaison.md / .csv / .json
Visuels qualitatifs	🌀 Partiel	3 images annotées par le seul modèle pré-entraîné dans data/outputs/ — il manque les mêmes images passées dans le fine-tuné
Application Streamlit	✗ À faire	Aucun dossier app/ ; le guide contient le code
Article LaTeX	✗ À faire	Trame prête dans le guide
Slides mis à jour	✗ À faire	Le rapport v2 contient encore des attentes, pas des résultats

Le point bloquant absolu : `results/` est vide. Sans le tableau comparatif, il n'y a **ni article, ni slides, ni onglet chiffres dans l'application**. Tout le reste en dépend.

Si le run a été fait sur Colab, les fichiers sont dans le Drive `MyDrive/Projet1_AMA/results/` : téléchargez-les et déposez-les dans `results/`. Sinon, relancez le notebook, ou en local :

```
python scripts/compare_models.py
```

Avant toute autre chose.

30. L'ordre des opérations d'ici la soutenance

1. **Récupérer ou produire** `results/comparaison.{md,csv,json}` et `results/eval_*.json`. Rien d'autre ne peut commencer avant.
2. **Choisir l'image de démonstration**. Une seule, la plus parlante : un embouteillage avec beaucoup de véhicules au fond. La passer dans les deux modèles avec `detect_image.py`, flouter plaques et visages.
3. **Remplacer chaque affirmation du rapport technique par un chiffre**. Diapositive 5 en priorité : ses puces sont des impressions tant que l'évaluation du pré-entraîné n'est pas citée. C'est la première chose qu'un jury technique attaquera.
4. **Rédiger l'article** (prompt fourni dans `PROMPTS_CLAUDE.md`).
5. **Construire l'application Streamlit** (prompt fourni également).
6. **Répéter la soutenance à trois**, chronomètre en main, avec le plan de `PLAN_SOUTENANCE.md`.

Règle de cohérence non négociable : tous les chiffres cités dans l'article, les slides et l'application doivent provenir **d'un seul et même run**, celui que le groupe désigne comme définitif. Mélanger deux exécutions dans un même tableau serait incohérent et se verrait.

31. Test de Feynman : sept phrases à savoir dire sans notes

Si vous pouvez énoncer ces sept phrases de tête, sans les lire, vous maîtrisez le projet.

1. **Le problème.** « YOLOv8 rate les véhicules qui occupent peu de pixels, parce que l'architecture du réseau réduit progressivement la résolution : un objet de 20 pixels disparaît des couches profondes. En trafic dense béninois, ces véhicules sont majoritaires. »
2. **La méthode.** « Nous avons fine-tuné YOLOv8n sur 2 400 images de trafic urbain dense, puis mesuré l'écart avec le modèle d'origine sur exactement le même jeu de test de 300 images. »
3. **La précaution n° 1.** « Nos quatre classes sont alignées sur COCO, et les labels du test sont réindexés à la volée dans l'espace de classes de chaque modèle. Sans cela, la comparaison porterait sur des étiquettes différentes et les métriques du pré-entraîné seraient fausses. »
4. **La précaution n° 2.** « Le rappel est ventilé par taille selon la convention COCO, avec les aires ramenées à la résolution d'entrée du réseau. Sans cette normalisation, toutes nos boîtes tombaient dans "grand" et la mesure ne mesurait rien — c'est ce qui s'est produit à notre premier essai. »
5. **La précaution n° 3.** « Le jeu de test est isolé : BMD-45 n'en fournit pas, nous l'avons dérivé de la validation de façon déterministe avec la graine 42. Sans test isolé, il n'y a pas de résultat, seulement une démonstration. »
6. **Le résultat.** « La ligne décisive est le rappel sur les objets petits : c'est elle qui répond au titre du projet. » *(puis*

votre chiffre réel)

7. **La limite.** « Aucun jeu public béninois n'existe. Nous avons travaillé sur du trafic dense de contexte comparable, avec 56 % de deux-roues dans nos annotations. La validation sur données locales est la suite du travail. »

32. Glossaire express

Terme	Traduction en une ligne
Bounding box	Rectangle entourant un objet
Backbone / Neck / Head	Extraction de caractéristiques / fusion multi-échelle / prédiction
Stride	Facteur de réduction de résolution d'une carte de caractéristiques
Anchor-free	Prédire la boîte directement, sans partir de rectangles types
IoU	Recouvrement entre deux boîtes, entre 0 et 1
NMS	Suppression des détections en doublon
VP / FP / FN	Détection juste / fausse alerte / objet manqué
Précision	Part des détections qui sont justes
Rappel	Part des objets réels effectivement trouvés
F1-score	Moyenne harmonique de précision et rappel
AP / mAP	Aire sous la courbe précision-rappel / sa moyenne sur les classes
mAP@0.5:0.95	Moyenne des mAP pour des seuils d'IoU de 0,50 à 0,95
Fine-tuning	Poursuivre l'entraînement d'un modèle existant sur de nouvelles données
Domain gap	Chute de performance quand les données de test diffèrent de celles d'entraînement
Epoch / Batch	Un passage sur tout le jeu / un paquet d'images traité en une fois
Overfitting	Le modèle apprend par cœur au lieu de généraliser
Early stopping	Arrêter quand la validation ne progresse plus
Seed	Graine du hasard, garantit la reproductibilité
FPS	Images traitées par seconde
Mosaic	Augmentation assemblant 4 images en une
ANPR / OCR	Lecture automatique de plaque / reconnaissance de texte (hors périmètre)